

Compiler Project

This project is a simple Compiler Construction tool for a Tiny Programming Language.
It demonstrates the first phase of a compiler, which is Lexical Analysis, by transforming a given language specification into formal models and a working GUI tool.

Name	Section
Zeyad Yasser Salah	3
Seif-Eldeen Taha	3
Seif Mujahid	3
Zeyad Magdy	3
Ibrahim Mokther	1

Under Supervision: Eng Crestien

1. Language Tokens

a. Reserved Words

- if
 - else
 - then
 - end
 - repeat
 - until
 - read
 - write
-

b. Operators

- Arithmetic Operators: +, -, *, /
 - Relational Operators: <, >, <=, >=
 - Assignment Operator: :=
-

c. Symbols

- ;
 - (
 -)
-

d. Identifiers

- Definition:
Identifier $\rightarrow$ Letter (Letter | Digit)*
-

e. Numbers

- Definition:
Number $\rightarrow$ digit+
or
digit (digit)*

f. Comments

- Definition:
 $\backslash[^\backslash]^*\backslash$

2. Language Definition using Regular Expressions

This section defines the lexical structure of the language using regular expressions. It describes how different tokens such as identifiers, numbers, keywords, operators, symbols, and comments are formed.

Identifier

Identifier $\rightarrow$ letter (letter | digit)*

- Starts with a letter.
 - Can be followed by zero or more letters or digits.
-

Number

Number $\rightarrow$ digit+

- Consists of one or more digits.
 - Represents integer values.
-

Keyword

Keyword $\rightarrow$ if | then | else | end | repeat | until | read | write

- Reserved words with predefined meanings in the language.
- Cannot be used as identifiers.

Operator

Operator $\rightarrow + \mid - \mid * \mid / \mid = \mid < \mid :=$

- Includes arithmetic, relational, and assignment operators.

Symbol

Symbol $\rightarrow ; \mid (\mid) \mid ,$

- Special characters used for structuring the program.

Assignment

Assignment $\rightarrow :=$

- Used to assign values to variables.

Comment

Comment $\rightarrow \backslash \{ [^] \}^* \backslash \}$

Explanation of the Comment Regular Expression

The comment is defined as:

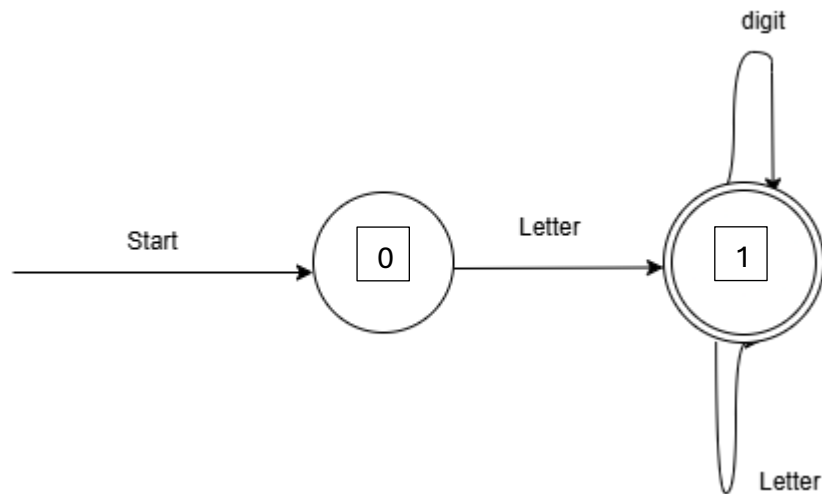
$\backslash \{ [^] \}^* \backslash \}$

Breakdown:

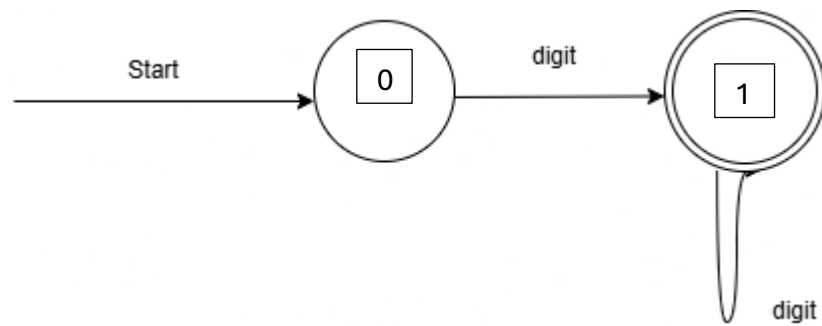
- $\backslash \{ \rightarrow$ Matches the opening curly brace {
- $[^] \rightarrow$ Matches any character except }

- * → Means "zero or more repetitions"
- \} → Matches the closing curly brace }

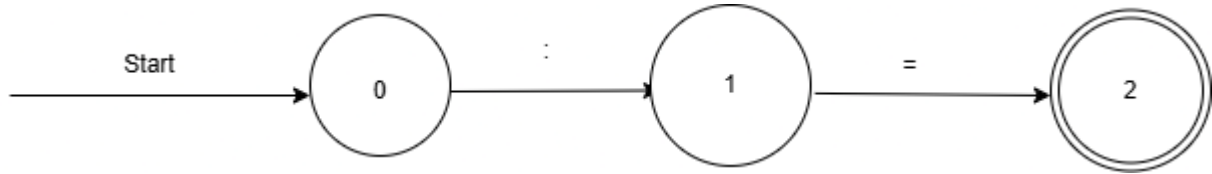
NFA For Identifier:



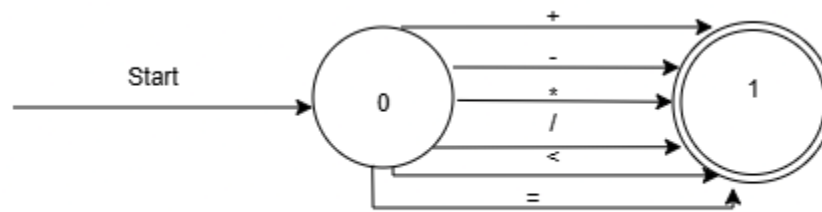
NFA For Number:



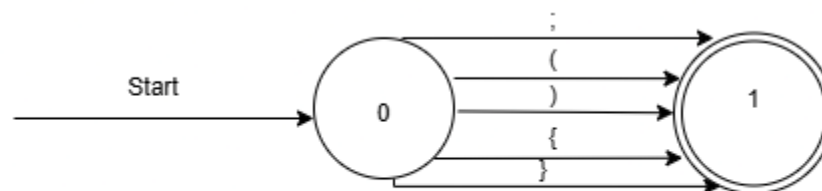
NFA For Assign Operator:



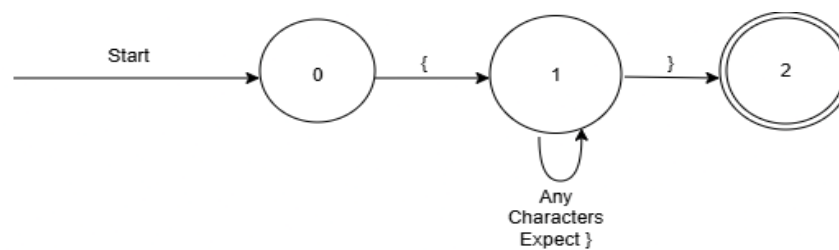
Nfa For Operators :



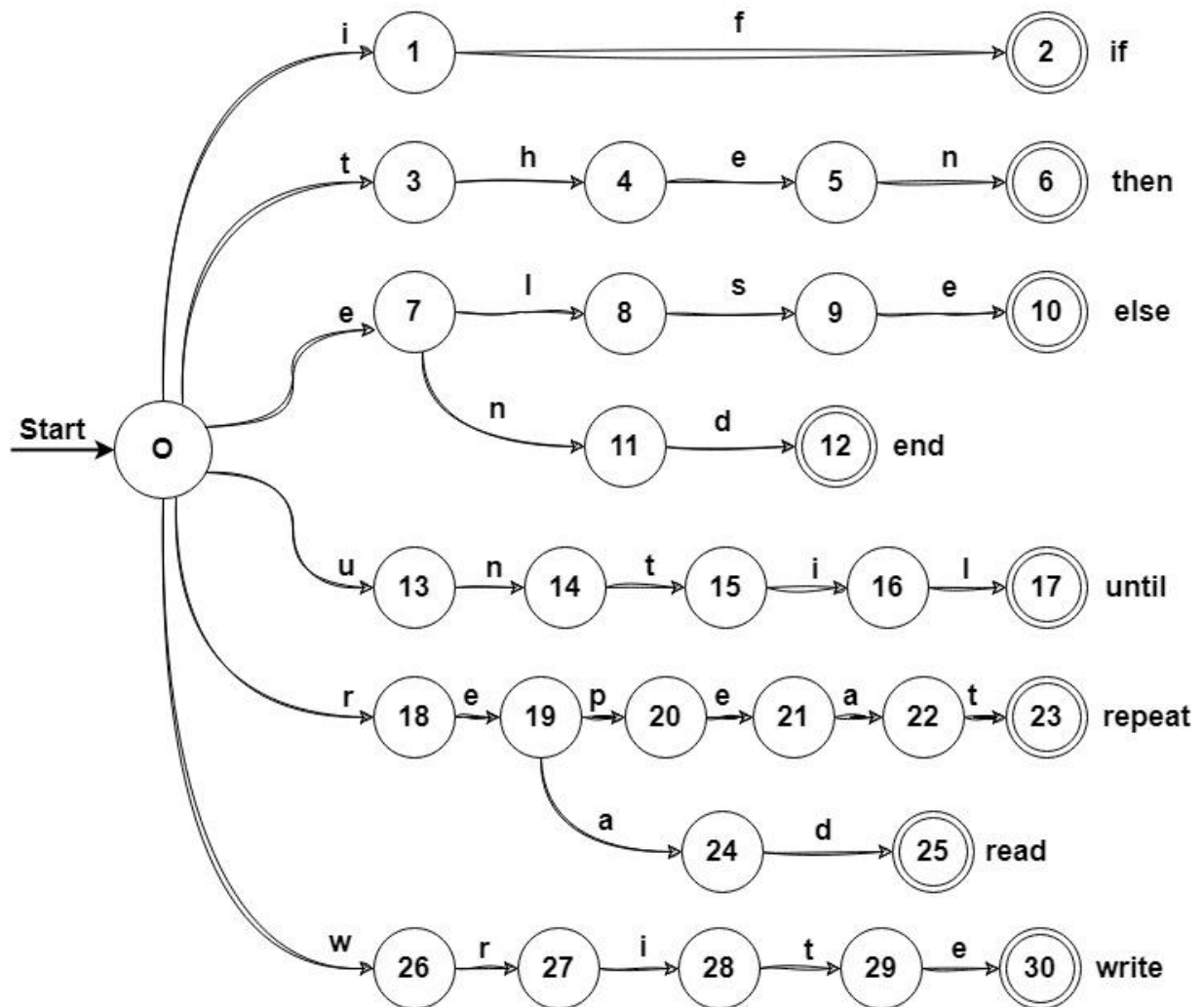
NFA for Symbols :



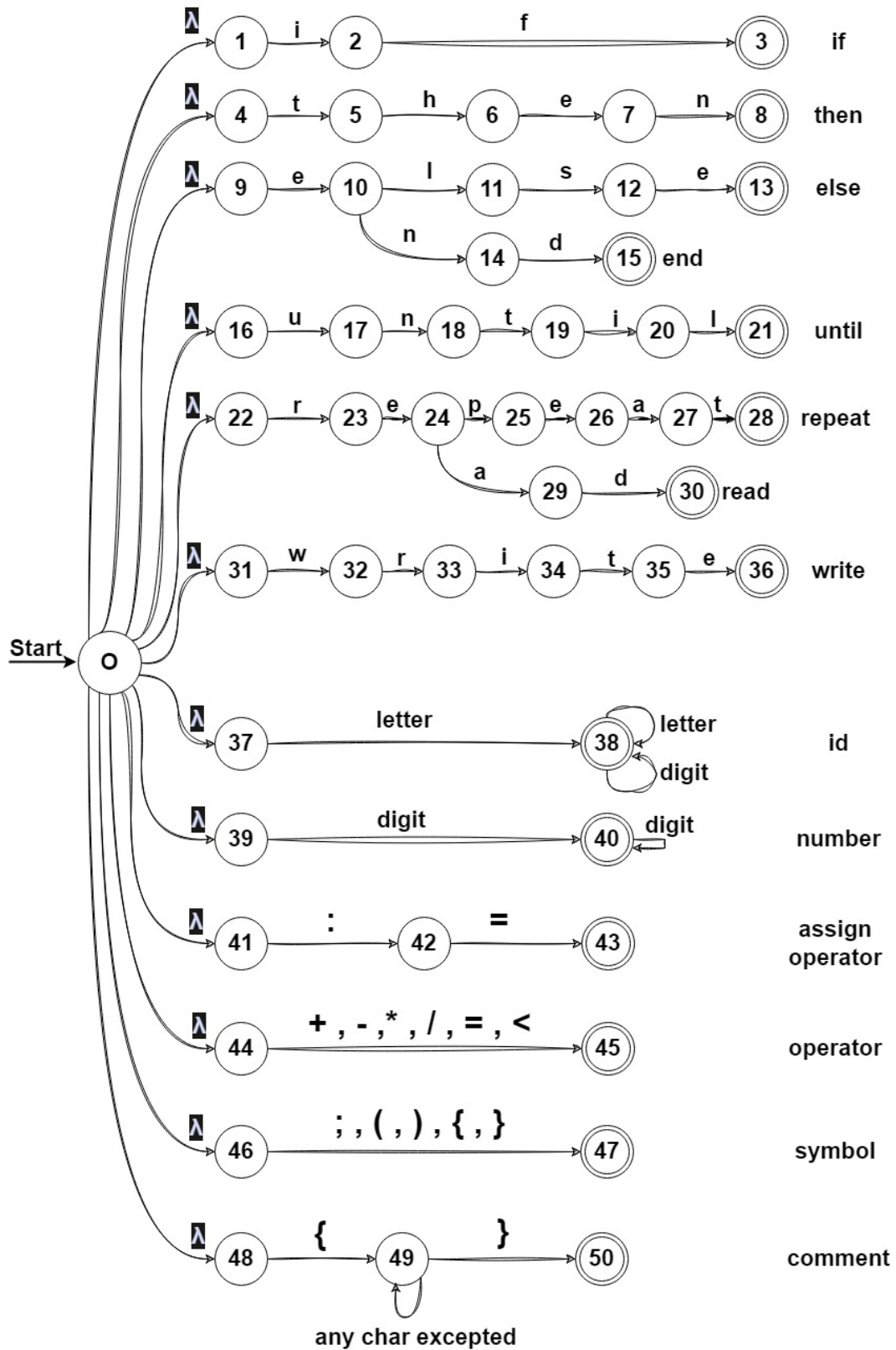
NFA For Comment :



NFA For Keyword:



Full NFA:



Convert NFA TO DFA

[illegible]

[illegible]

Link to Convert Nfa to Dfa : [NFA.xlsx](#)

DFA Table:

Link to [DFA_full.xlsx](#)

Draw DFA

Lexical Analyzer Implementation (GUI-based)

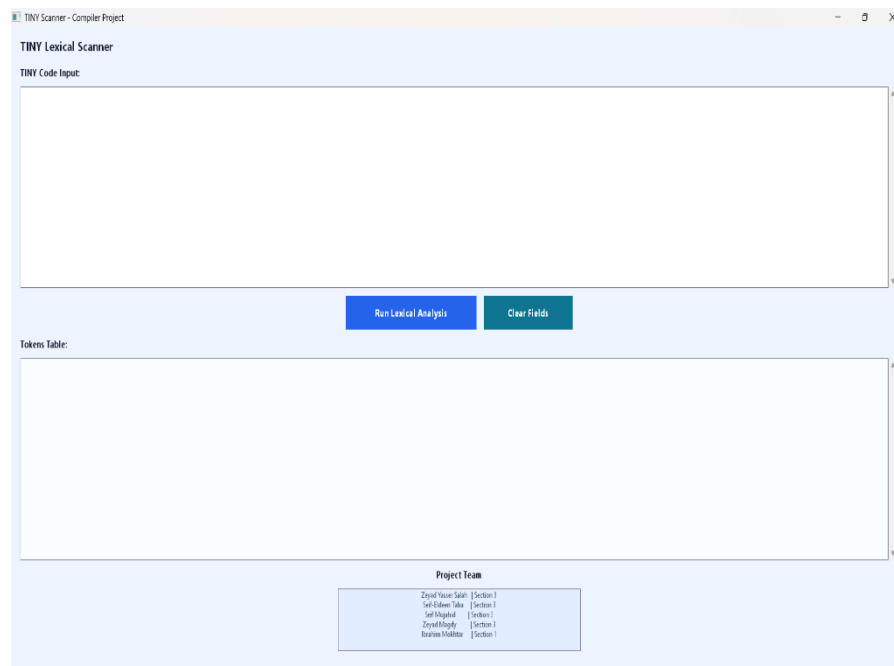
A simple graphical user interface that allows users to:

- Enter source code in a text area
- Click **Run** to perform lexical analysis
- Click **Clear** to reset the input and output fields

How to Run:

```
- cd .\Project\  
- g++ "mainwindow.cpp" -o "mainwindow.exe" -mwindows -lgdi32 -luser32  
- .\mainwindow.exe
```

Screen :



Test Code :



```
1  { scanner full test }
2  read x;
3  read y;
4  if x <= 10 then
5      z := x + y * 2;
6  else
7      z := y - 3 / 1;
8  end
9  repeat
10     x := x + 1;
11 until x >= y
12 write z, x;
13 if x > y then
14     write x;
15 end
16
```

Results :

TINY Scanner - Compiler Project

TINY Lexical Scanner

TINY Code Input:

```
read x;
read y;
if x <= 10 then
  z := x + y * z;
else
  z := y - 3 / 1;
end
repeat
  x := x + 1;
until x >= y
write z, x;
if x > y then
  write x;
end
```

[Run Lexical Analysis](#) [Clear Fields](#)

Tokens Table:

Token#	Lexeme	Type
1	read	READ
2	x	IDENTIFIER
3	;	SEMICOLON
4	read	READ
5	y	IDENTIFIER
6	;	SEMICOLON
7	if	IF
8	x	IDENTIFIER
9	<=	COMPARISON
10	10	NUMBER
11	then	THEN
12	z	IDENTIFIER
13	:=	ASSIGNMENT

Project Team

Zeyad Yasser Salah	Section 3
Said Hossam Taha	Section 3
Said Maghadd	Section 3
Zeyad Magdy	Section 3
Brabson Muhitar	Section 1

GitHub Repo: <https://github.com/zeyadyasser228/Compiler-project>

Conclusion

This project successfully demonstrates the fundamental concepts of **compiler construction**, specifically the **lexical analysis phase** for a Tiny Programming Language. By defining the language using **regular expressions** and converting them into **NFA and DFA**, the system shows how theoretical concepts are applied in practical compiler design.

The implementation of the **GUI-based lexical analyzer** provides a user-friendly way to test source code input and observe token recognition in real time. It efficiently handles different types of lexical elements such as identifiers, numbers, keywords, operators, and comments.

Overall, the project bridges the gap between theory and practice, helping to understand how compilers process programming languages at the initial stage and how formal language concepts are used to build real-world tools.